Geometric Transformations of Images

Formulas, matrices, explanations and a practice quiz

These notes collect the mathematics behind the notebook `3D_CV_Lab2.ipynb` (affine and perspective transformations with OpenCV): image coordinates, homogeneous coordinates, translation, scaling, rotation, shear, homographies, composition of transformations, and how a warp is actually computed pixel by pixel.

How to use these notes **Part I** explains every formula in the order the notebook introduces it. Each section ends with an *In the notebook* box that links the maths to the code. **Part II** is a 41-question multiple-choice quiz grouped by topic. **Part III** has the answer key and a worked explanation for every question (click a question number to jump between a question and its solution). The last page is a **one-page formula sheet** for quick revision.

Conventions used throughout Points are written as (x, y) with x = column (to the right) and y = row (downwards), exactly as OpenCV does. Angles in formulas are in radians unless marked with $^\circ$. Matrices act on column vectors: $\mathbf{p}' = M\mathbf{p}$.

Contents

Part I Theory: formulas and explanations 2

1 Images as arrays: coordinates and conventions 2

2 Homogeneous coordinates 2

3 Translation 3

4 Scaling 4

5 Rotation 5

6 Shear (skew) 6

7 The affine family 7

8 Perspective transformation (homography) 8

9 Composing transformations 9

10 Warping in practice: mapping and interpolation 11

Part II Practice quiz 13

Part III Solutions 18

Formula sheet 22

Part I

Theory: formulas and explanations

1 Images as arrays: coordinates and conventions

A geometric transformation changes *where* each pixel is, not its colour. Before we can write such a rule as a formula we need to agree on how a pixel's position is described.

A colour image of width W and height H is stored by OpenCV as a NumPy array of shape $(H, W, 3)$. Every entry is an 8-bit integer (`uint8`) from 0 to 255.

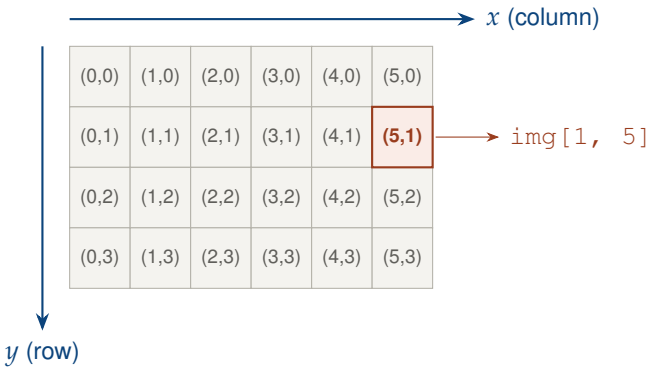


Figure 1. Pixel coordinates in OpenCV. The origin is the top-left pixel, x grows to the right and y grows *downwards*. This tiny image has `img.shape = (4, 6, 3)`.

- **Origin.** $(0, 0)$ is the *top-left* pixel; the last pixel is $(W-1,\ H-1)$.
- **Axes.** x increases to the right (column index), y increases *downwards* (row index).
- **Indexing.** NumPy indexes row first: the pixel at (x, y) is `img[y, x]`.
- **Sizes in OpenCV calls** are written as `(W, H)`, for example `dsize = (img.shape[1], img.shape[0])`.
- **Channel order.** OpenCV stores colours as B, G, R. Matplotlib expects R, G, B, hence `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` before `plt.imshow`.

Four different orders Maths writes (x, y); NumPy indexing is `img[y, x]`; the shape is $(H, W, 3)$; OpenCV size arguments are (W, H). Mixing these up is the most common source of bugs. Because y points down, rotations look mirrored compared with a school-maths drawing (see Section 5).

2 Homogeneous coordinates

Homogeneous coordinates are the trick that lets every transformation in the notebook, including translation, be written as a single matrix multiplication.

2.1 Why a 2×2 matrix is not enough

A 2×2 matrix A multiplies coordinates: $A \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a_{11}x + a_{12}y \\ a_{21}x + a_{22}y \end{bmatrix}$. Every term contains x or y, so there is no place for a constant. In particular

$$A \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \text{for every } A,$$

whereas a translation must move the origin to $(t_x, t_y) \neq (0,0)$. Hence translation is *not* a linear map of (x, y) and cannot be a 2×2 matrix.

2.2 The trick: add a third coordinate

Write the point (x, y) as the 3-vector $\tilde{\mathbf{p}} = (x,\ y,\ 1)^\mathsf{T}$. More generally,

$$(X,\ Y,\ W)^\mathsf{T} \ \text{ with } W \neq 0 \quad \text{represents the 2D point} \quad \left(\frac{X}{W},\ \frac{Y}{W}\right),$$

so $(X, Y, W)^\mathsf{T}$ and $(\lambda X, \lambda Y, \lambda W)^\mathsf{T}$ (any $\lambda \neq 0$) are the *same* point. Example: $(6, 4, 2)^\mathsf{T} \sim (3, 2, 1)^\mathsf{T}$, both mean the point $(3, 2)$.

Every transformation in this notebook is a 3×3 matrix acting on $\tilde{\mathbf{p}}$:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \qquad x_{\text{final}} = \frac{x'}{w'}, \quad y_{\text{final}} = \frac{y'}{w'}.$$

Key idea The third column multiplies the constant 1, so it is the "add a constant" slot that makes translation possible. The bottom row decides whether the division by w' does anything. If the bottom row is $(0, 0, 1)$ then $w' = 1$ for every point and the transformation is called **affine**:

$$M_{\text{affine}} = \begin{bmatrix} A & \mathbf{t} \\ \mathbf{0}^\mathsf{T} & 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{p}' = A\mathbf{p} + \mathbf{t}.$$

In the notebook `cv2.warpAffine(img, M, dsize)` takes only the top two rows of an affine matrix (a 2×3 array), because the bottom row is always $(0, 0, 1)$. `cv2.warpPerspective(img, H, dsize)` takes the full 3×3 matrix and performs the division by w'.

3 Translation

$$T(t_x, t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}, \qquad T \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}.$$

Reading the product row by row: row 1 gives $1 \cdot x + 0 \cdot y + t_x \cdot 1 = x + t_x$; row 2 gives $y + t_y$; row 3 keeps the 1.

- **Inverse:** $T(t_x, t_y)^{-1} = T(-t_x, -t_y)$.
- **Composition:** $T(\mathbf{a})\,T(\mathbf{b}) = T(\mathbf{a}+\mathbf{b})$; translations commute with each other.
- **Preserves** lengths, angles, areas and orientation. Degrees of freedom: 2.

Worked example $(10, 20)$ with $\mathbf{t} = (100, 50)$ goes to $(110, 70)$. With an output canvas of the original size, a shift by $t_x > 0$ leaves a black strip of width t_x on the *left* (no source pixel lands there) and cuts off t_x columns on the right. A negative t_x does the opposite; $t_y > 0$ moves the image *down* because y points down.

In the notebook

```
T = np.array([[1, 0, tx],
              [0, 1, ty]], dtype=np.float32)          # top two rows of T(tx, ty)
cv2.warpAffine(img, T, (img.shape[1], img.shape[0]))  # dsize = (W, H)
```

4 Scaling

$$S(s_x, s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad (x, y) \;\mapsto\; (s_x x,\; s_y y).$$

- $s > 1$ enlarges, $0 < s < 1$ shrinks. $s_x = s_y$ is *uniform* scaling (shape kept); $s_x \neq s_y$ changes the aspect ratio; a negative factor mirrors the image.
- **Fixed point:** only the origin $(0, 0)$, the top-left corner. Every other point moves along the ray from the origin, and its distance from the origin is multiplied by s. The image therefore grows towards the bottom-right, not from its centre.
- **Inverse:** $S(1/s_x,\, 1/s_y)$. **Determinant:** $s_x s_y$ = the factor by which areas change.
- **Output size** needed to hold the whole result: $\big(\lfloor W s_x \rfloor,\; \lfloor H s_y \rfloor\big)$.
- **Scaling about a point c** (see Section 9):

$$T(\mathbf{c})\,S\,T(-\mathbf{c}) = \begin{bmatrix} s_x & 0 & (1-s_x)c_x \\ 0 & s_y & (1-s_y)c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

Worked example $S(2, 2)$ sends $(30, 10)$ to $(60, 20)$: same direction from the origin, twice as far. An image with `shape (400, 600, 3)` scaled by 2.5 needs `dsize = (1500, 1000)` and the result has `shape (1000, 1500, 3)`.

In the notebook `scale_image_warp` uses `[[sx,0,0],[0,sy,0]]` and `new_size = (int(W*sx), int(H*sy))`. Keeping the original size instead would show only the top-left $1/s$ of an enlarged image. In practice `cv2.resize` is the usual way to scale; the notebook uses `warpAffine` to show that scaling is "just another matrix". Where the new pixel values come from is explained in Section 10.

5 Rotation

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

$$x' = x\cos\theta - y\sin\theta, \qquad y' = x\sin\theta + y\cos\theta.$$

5.1 Derivation

Describe the point by its distance r from the origin and its angle φ: $x = r\cos\varphi$, $y = r\sin\varphi$. A rotation keeps r and adds θ to the angle. Using the angle-addition formulas,

$$x' = r\cos(\varphi+\theta) = r\cos\varphi\cos\theta - r\sin\varphi\sin\theta = x\cos\theta - y\sin\theta,$$
$$y' = r\sin(\varphi+\theta) = r\sin\varphi\cos\theta + r\cos\varphi\sin\theta = x\sin\theta + y\cos\theta.$$

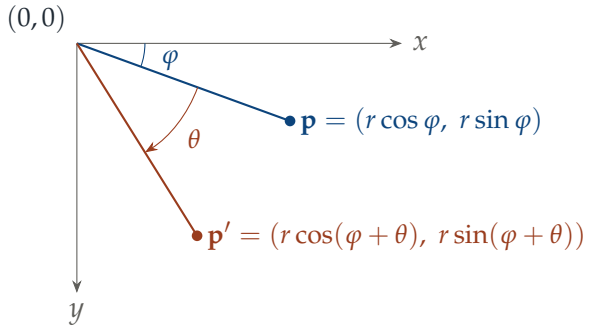


Figure 2. Rotation keeps the distance r and adds θ to the angle. Because y points down, a positive angle turns *clockwise* on screen.

Column view (works for every matrix). The columns of the 2×2 block are the images of the unit vectors: $(1,0) \mapsto (\cos\theta, \sin\theta)$ and $(0,1) \mapsto (-\sin\theta, \cos\theta)$. Knowing where these two arrows go determines the map.

5.2 Properties

$$R(\theta)^{\mathsf{T}} R(\theta) = I, \qquad \det R(\theta) = \cos^2\theta + \sin^2\theta = 1,$$
$$R(\theta)^{-1} = R(\theta)^{\mathsf{T}} = R(-\theta), \qquad R(\alpha)R(\beta) = R(\alpha+\beta).$$

Rotations preserve lengths, angles and areas. Rotation plus translation is a *rigid* (Euclidean) motion with 3 degrees of freedom.

Direction and pivot **Direction.** With $\theta = 90°$ the point $(1,0)$ (to the right of the origin) goes to $(0,1)$, which is *below* the origin in an image. Right $\rightarrow$ down is clockwise on screen, so a positive θ looks clockwise.
Pivot. $R(\theta)$ rotates about $(0,0)$, the top-left corner. For $\theta = 90°$ we get $x' = -y \leq 0$ for every pixel, so practically the whole image leaves the canvas and the output is black (at most one column survives).

Units. `np.cos/np.sin` expect radians: $\theta_{\text{rad}} = \theta_{\text{deg}} \cdot \pi/180$. `np.cos(10)` is the cosine of 10 radians ($\approx 573°$), about -0.839.

Rotation about a point c.

$$T(\mathbf{c})\, R(\theta)\, T(-\mathbf{c}) = \begin{bmatrix} R_{2\times 2} & \mathbf{c} - R_{2\times 2}\,\mathbf{c} \\ \mathbf{0}^{\mathsf{T}} & 1 \end{bmatrix}.$$

OpenCV's helper `cv2.getRotationMatrix2D(center, angle, scale)` returns

$$\begin{bmatrix} \alpha & \beta & (1-\alpha)c_x - \beta c_y \\ -\beta & \alpha & \beta c_x + (1-\alpha)c_y \end{bmatrix}, \qquad \alpha = s\cos\theta,\ \beta = s\sin\theta,$$

which is $T(\mathbf{c})\,\big(s\,R(-\theta)\big)\,T(-\mathbf{c})$: in OpenCV's helper a *positive angle means counter-clockwise* on screen, the opposite of the notebook's `rotate_image`.

In the notebook `rotate_image(img, angle)` converts degrees with `np.deg2rad` and builds `[[cos,-sin,0],[sin,cos,0]]`. So `rotate_image(img, 10)` turns the image 10° clockwise about its top-left corner; black wedges open along the top and right edges and the lower-left part swings off the canvas.

6 Shear (skew)

$$H_x(k) = \begin{bmatrix} 1 & k & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} : (x, y) \mapsto (x + ky,\ y), \qquad H_y(k) = \begin{bmatrix} 1 & 0 & 0 \\ k & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} : (x, y) \mapsto (x,\ y + kx).$$

In a horizontal shear each row slides sideways by $k \cdot y$: the top row ($y = 0$) is fixed and the bottom row moves the most. A rectangle becomes a parallelogram. Italic letters are sheared upright letters.

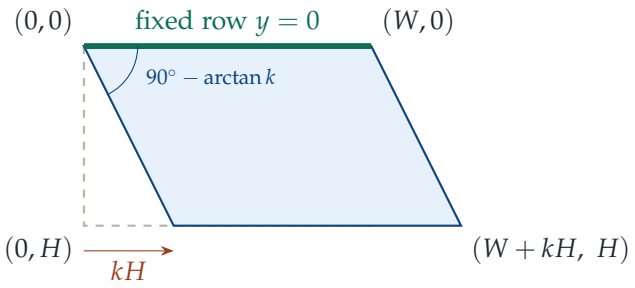


Figure 3. Horizontal shear ($k = 0.5$). Dashed: original image; filled: sheared image.

- **Columns:** $(1,0) \mapsto (1,0)$ (horizontal lines stay horizontal); $(0,1) \mapsto (k,1)$ (vertical lines lean).
- $\det H_x(k) = 1$: **area is preserved**. Straight lines stay straight and parallel lines stay parallel.
- **Angles are not preserved:** the right angle at a corner becomes $90° - \arctan k$ (e.g. $k = 0.3 \Rightarrow 73.3°$).

- **Inverse:** $H_x(-k)$.
- **Extent** (for $k \geq 0$): width $W + kH$, height unchanged. A tight canvas is $(W + kH,\ H)$. For $k < 0$ part of the image lands at negative x and is lost unless a translation is added.

> **In the notebook** `shear_image(img, k)` uses `[[1,k,0],[0,1,0]]` and a canvas of `(2*W, 2*H)`. Doubling the width is enough as long as $kH \leq W$; doubling the height only adds a black band, because a horizontal shear never changes y.

7 The affine family

A general affine transformation has 6 free numbers:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}.$$

Each point correspondence $(x, y) \leftrightarrow (x', y')$ gives 2 equations, so **3 non-collinear point pairs** determine it (`cv2.getAffineTransform`). The linear part can always be built from the building blocks above, e.g. $A = R(\theta)\, H_x(k)\, S(s_x, s_y)$ (with a negative scale factor if a reflection is needed).

> **Parallelograms stay parallelograms** An affine map sends a parallelogram to a parallelogram. If corners A, B, C go to A', B', C', the fourth corner $D = B + C - A$ must go to $D' = B' + C' - A'$. This is why 3 points fix an affine map, and why moving a single corner of a rectangle independently needs a perspective transform.

Building block	Lengths	Angles	Parallel lines	Area
Translation	kept	kept	kept	kept
Rotation	kept	kept	kept	kept
Uniform scaling	$\times s$	kept	kept	$\times s^2$
Non-uniform scaling	changed	changed	kept	$\times s_x s_y$
Shear	changed	changed	kept	kept
Perspective (Section 8)	changed	changed	**not kept**	changed

Class	Matrix form	DOF	Invariants
Rigid (Euclidean)	$\begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$	3	lengths, angles, area
Similarity	$\begin{bmatrix} sR & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$	4	angles, ratios of lengths (shape)
Affine	$\begin{bmatrix} A & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$	6	parallelism, ratios along a line, area ratios
Projective (homography)	$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix}$	8	straight lines (collinearity), cross-ratio

Each class contains the ones above it: rigid $\subset$ similarity $\subset$ affine $\subset$ projective.

8 Perspective transformation (homography)

8.1 The matrix and the division

A photo of a flat rectangle taken at an angle is a general quadrilateral whose parallel edges seem to converge. No affine map can do that, so the bottom row is freed:

$$H = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix}, \qquad \begin{aligned} x' &= ax + by + c \\ y' &= dx + ey + f \\ w' &= gx + hy + 1 \end{aligned} \qquad x_{\text{final}} = \frac{ax+by+c}{gx+hy+1}, \quad y_{\text{final}} = \frac{dx+ey+f}{gx+hy+1}.$$

Now w' depends on the position. Where w' is large the point is divided by a big number and pulled towards the origin, so that region shrinks and looks farther away. This position-dependent scale is what creates the impression of depth.

Worked example: one number makes a vanishing point Take the identity and set only $g = 0.002$, so $w' = 1 + 0.002x$:

$$(0, 100) \mapsto (0, 100), \qquad (500, 100) \overset{w'=2}{\longmapsto} (250, 50), \qquad (500, 200) \overset{w'=2}{\longmapsto} (250, 100).$$

The lines $y = 100$ and $y = 200$ are 100 px apart at $x = 0$ but only 50 px apart at $x = 500$: they converge. Along any horizontal line, $x_{\text{final}} = \frac{x}{1+0.002x} \to \frac{1}{g} = 500$ and $y_{\text{final}} = \frac{y}{1+0.002x} \to 0$ as $x \to \infty$, so *all* horizontal lines meet at the vanishing point $(500, 0)$. Vertical lines stay vertical here because x_{final} depends only on x.

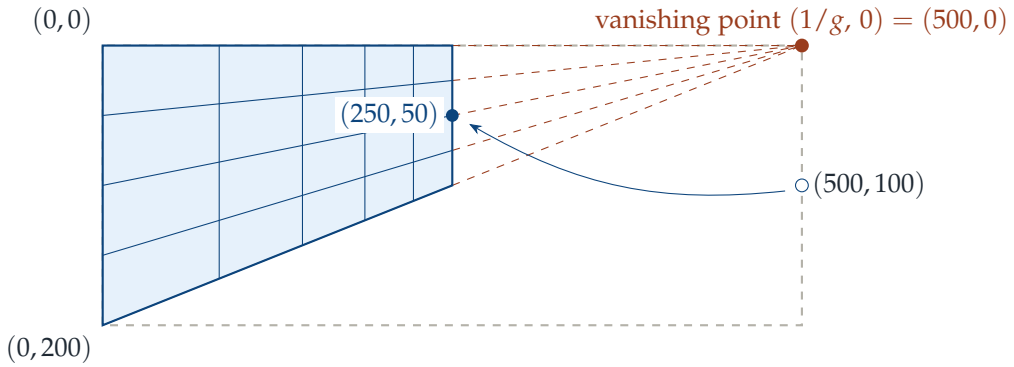


Figure 4. The rectangle $[0, 500] \times [0, 200]$ (dashed) under the homography with $g = 0.002$ (filled). Horizontal grid lines converge towards the vanishing point.

Points at infinity (why affine keeps parallels). A direction (d_x, d_y) is the homogeneous point $(d_x, d_y, 0)^\mathsf{T}$: a "point at infinity" where all parallel lines with that direction meet. An affine matrix keeps the third entry 0, so parallel lines still meet at infinity, i.e. they stay parallel. A homography maps it to $(\ldots, \ldots, gd_x + hd_y)^\mathsf{T}$; when $gd_x + hd_y \neq 0$ this is a finite point: the vanishing point.

8.2 Why 8 unknowns and 4 point pairs

H and λH ($\lambda \neq 0$) give the same final points because λ cancels in the division. So one entry can be fixed ($h_{33} = 1$), leaving **8 degrees of freedom**. For one pair $(x, y) \leftrightarrow (u, v)$,

$$u = \frac{ax + by + c}{gx + hy + 1}, \qquad v = \frac{dx + ey + f}{gx + hy + 1}.$$

Multiplying both sides by the denominator gives two equations that are *linear* in the unknowns:

$$\begin{aligned} ax + by + c - gxu - hyu &= u \\ dx + ey + f - gxv - hyv &= v \end{aligned} \iff \begin{bmatrix} x & y & 1 & 0 & 0 & 0 & -xu & -yu \\ 0 & 0 & 0 & x & y & 1 & -xv & -yv \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \\ g \\ h \end{bmatrix} = \begin{bmatrix} u \\ v \end{bmatrix}.$$

Four pairs (no three collinear) give an 8×8 linear system with a unique solution. This is the Direct Linear Transform (DLT) idea used by `cv2.getPerspectiveTransform`. With many noisy correspondences, `cv2.findHomography(src, dst, cv2.RANSAC)` fits the best H instead. (Compare: an affine map has 6 unknowns, so it needs only 3 pairs.)

- Lines map to lines; parallel lines may meet; lengths, angles and areas generally change.
- The inverse H^{-1} is again a homography.
- A homography is the exact relation between two photos of a *plane* (or two photos taken from the same camera centre), which is why it is everywhere in 3D vision: document scanning, bird's-eye views, panorama stitching, augmented reality.

In the notebook

```
src_pts = np.float32([[0, 0], [w-1, 0], [0, h-1], [w-1, h-1]])  # TL, TR, BL, BR
dst_pts = np.float32([[50, 50], [w-50, 0], [0, h-50], [w, h]])   # same order!
T = cv2.getPerspectiveTransform(src_pts, dst_pts)  # solves the 8x8 system
cv2.warpPerspective(img, T, (w, h))                # divides by w'
```

The top-left corner is pushed $(50, 50)$ diagonally inward, the top-right moves about 50 px left, the bottom-left about 50 px up and the bottom-right barely moves. For a 600×400 image the solved matrix is approximately

$$T \approx \begin{bmatrix} 0.622 & -0.125 & 50 \\ -0.0835 & 0.623 & 50 \\ -0.000387 & -0.000369 & 1 \end{bmatrix};$$

note the small but non-zero g, h in the bottom row.

9 Composing transformations

9.1 Right to left

Applying A first and then B:

$$\mathbf{p}' = B\,(A\,\mathbf{p}) = (BA)\,\mathbf{p}.$$

The **rightmost** matrix acts first. A whole chain collapses into one matrix $M = M_n \cdots M_2 M_1$, which is the main payoff of homogeneous coordinates. To undo a chain, undo the steps in reverse order: $(BA)^{-1} = A^{-1}B^{-1}$.

9.2 Order matters

Matrix multiplication is not commutative. For rotation and translation:

$$\text{rotate, then translate:} \qquad T(\mathbf{t})\,R(\theta) = \begin{bmatrix} R_{2\times 2} & \mathbf{t} \\ \mathbf{0}^\mathsf{T} & 1 \end{bmatrix},$$

$$\text{translate, then rotate:} \qquad R(\theta)\,T(\mathbf{t}) = \begin{bmatrix} R_{2\times 2} & R_{2\times 2}\mathbf{t} \\ \mathbf{0}^\mathsf{T} & 1 \end{bmatrix}.$$

The 2×2 blocks agree; in the second case the shift vector itself is rotated.

Worked example $\theta = 10°$, $\mathbf{t} = (40, -30)$: $T\,R$ has third column $(40, -30)$, while $R\,T$ has third column $(0.985 \cdot 40 + 0.174 \cdot 30,\ 0.174 \cdot 40 - 0.985 \cdot 30) \approx (44.6,\ -22.6)$.

9.3 The pivot trick

To apply a linear transform X (rotation, scaling, shear) about a point $\mathbf{c}$ instead of the origin: move $\mathbf{c}$ to the origin, transform, move back:

$$T(\mathbf{c})\,X\,T(-\mathbf{c}) = \begin{bmatrix} X_{2\times 2} & \mathbf{c} - X_{2\times 2}\,\mathbf{c} \\ \mathbf{0}^\mathsf{T} & 1 \end{bmatrix}.$$

Examples: rotation about the image centre $\mathbf{c} = (W/2, H/2)$; uniform scaling by s about $\mathbf{c}$ has third column $(1 - s)\mathbf{c}$ (for $s = 3$, $\mathbf{c} = (100, 50)$: $(-200, -100)$).

Warp once, not several times Chaining `warpAffine` calls (i) interpolates at every step, so the image gets blurrier, and (ii) crops to the canvas at every step, so pixels lost in an intermediate result never come back (their black fill is moved into view instead). Multiply the matrices first and warp once. In NumPy the matrix product is `A @ B`; `A * B` is element-wise. Use `M[:2]` to pass the top two rows to `warpAffine`.

In the notebook

```
# cell 1: rotate, then translate (= T @ R), done with two separate warps
rotated_img = translate_image(rotate_image(img, 10), 40, -30)

# cell 2 as written: SyntaxError (extra ")"), missing tx, ty, result discarded
#   rotated_img = translate_image(img), 40,-30)
# intended: translate, then rotate (= R @ T)
rotated_img = rotate_image(translate_image(img, 40, -30), 10)

# better: build the matrices, multiply, warp once (rotation about the centre)
M = T_mat(cx, cy) @ R_mat(10) @ T_mat(-cx, -cy)
out = cv2.warpAffine(img, M[:2], (w, h))
```

`create_transformation` computes `T = t @ R @ Skew @ Scale`: read right to left, a point is first scaled, then skewed, then rotated, then translated. It then sets `T[2,0] = persp1` and `T[2,1] = persp2`, i.e. the entries g, h, which turns the affine matrix into a

> homography. Its `angle=0.81` is in radians ($\approx 46.4°$), unlike `rotate_image`, which takes degrees.

10 Warping in practice: mapping and interpolation

10.1 Forward vs. inverse mapping

Forward mapping: for every *source* pixel $\mathbf{p}$, compute $\mathbf{p}' = M\mathbf{p}$ and write `dst[p'] = src[p]`. Problems: when the map enlarges, some destination pixels are never hit (*holes*); when it shrinks, several source pixels land on the same destination pixel (*collisions*, the last write wins); positions must be rounded.

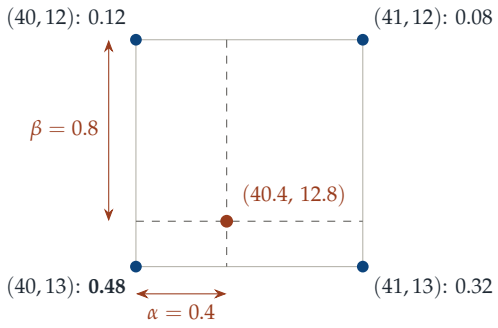
Inverse (backward) mapping: for every *destination* pixel $\mathbf{q}$, compute $\mathbf{p} = M^{-1}\mathbf{q}$ and set $\mathrm{dst}(\mathbf{q}) = \mathrm{src}(\mathbf{p})$, interpolating because $\mathbf{p}$ is usually not on the pixel grid. Every output pixel gets exactly one value. If $\mathbf{p}$ falls outside the source, the output gets the border value (black by default). `cv2.warpAffine` and `cv2.warpPerspective` work this way: they invert M internally (unless `flags` contains `cv2.WARP_INVERSE_MAP`).

10.2 Bilinear interpolation

Write the non-integer source position as $\mathbf{p} = (x_0 + \alpha,\ y_0 + \beta)$ with integers x_0, y_0 and $0 \le \alpha, \beta < 1$. Then

$$I(\mathbf{p}) = (1-\alpha)(1-\beta)\, I_{x_0,y_0} + \alpha(1-\beta)\, I_{x_0+1,y_0} + (1-\alpha)\beta\, I_{x_0,y_0+1} + \alpha\beta\, I_{x_0+1,y_0+1}.$$

The four weights add up to 1, and closer neighbours get larger weights (each weight equals the area of the sub-rectangle *opposite* that neighbour).



Worked example Enlarging by 2.5, output pixel $(101, 32)$ comes from $(101/2.5,\ 32/2.5) = (40.4,\ 12.8)$, so $\alpha = 0.4$, $\beta = 0.8$. Weights: $0.6 \cdot 0.2 = 0.12$, $0.4 \cdot 0.2 = 0.08$, $0.6 \cdot 0.8 = 0.48$, $0.4 \cdot 0.8 = 0.32$. Other options: `cv2.INTER_NEAREST` (copy the closest pixel, blocky), `cv2.INTER_CUBIC` (smoother), `cv2.INTER_AREA` (best for shrinking).

10.3 The hand-written loop in the notebook

`apply_transformation` loops over every source pixel and uses forward mapping:

```
p = np.array([[i], [j], [1]])       # i = ROW index, j = COLUMN index
new_p = T @ p
new_x = int(new_p[0, 0] / new_z)    # used as a row index
new_y = int(new_p[1, 0] / new_z)    # used as a column index
new_img[new_x, new_y] = img[i, j]
```

- **Swapped axes.** The vector is $(\mathrm{row}, \mathrm{col}, 1)$, not $(x, y, 1)$. The matrix's first coordinate is therefore *vertical*: `tx = 200` shifts the image *down* and `ty = 150` shifts it *right*, and a positive angle appears counter-clockwise on screen (swapping the axes mirrors the picture).

- **Forward mapping** leaves holes wherever the map locally enlarges, and overwrites pixels where it shrinks.
- `int()` truncates towards zero instead of rounding.
- A Python double loop runs $H \cdot W$ iterations, which is slow; OpenCV does the same work in optimised C++.

A vectorised inverse-mapping version (nearest neighbour, (x, y) convention, any 3×3 matrix):

```
Hinv = np.linalg.inv(T)
ys, xs = np.indices((h, w))                          # every destination pixel
pts = np.stack([xs.ravel(), ys.ravel(), np.ones(h * w)])
src = Hinv @ pts
src = src[:2] / src[2]                               # perspective division
sx, sy = np.rint(src).astype(int)                    # nearest neighbour
ok = (0 <= sx) & (sx < w) & (0 <= sy) & (sy < h)
out = np.zeros_like(img)
out.reshape(-1, 3)[ok] = img[sy[ok], sx[ok]]
```

Part II

Practice quiz

Each question has exactly one correct answer. Try them without looking back, then check Part III. Click a question number to jump to its solution.

A. Images and coordinates

Q1. `img = cv2.imread(...)` returns an array with `img.shape == (480, 640, 3)`. Which statement is correct?

- **A)** The image is 480 px wide and 640 px tall.
- **B)** The pixel at $x = 10$, $y = 20$ is `img[10, 20]`.
- **C)** The image is 640 px wide and 480 px tall, and the pixel at $x = 10$, $y = 20$ is `img[20, 10]`.
- **D)** The three channels are stored in R, G, B order.

Q2. In `cv2.warpAffine(img, M, dsize)`, the argument `dsize` must be given as

A) (width, height) **B)** (height, width) **C)** `img.shape` **D)** (width, height, 3)

Q3. Why does `show_image` call `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` before `plt.imshow`?

- **A)** Matplotlib can only display floating-point images.
- **B)** It converts the image to grayscale so it displays faster.
- **C)** `warpAffine` returns images in a different colour space.
- **D)** OpenCV stores channels as B, G, R, while Matplotlib expects R, G, B.

B. Homogeneous coordinates and translation

Q4. Why can't a translation $(x, y) \mapsto (x + t_x,\, y + t_y)$ with $\mathbf{t} \neq \mathbf{0}$ be written as a 2×2 matrix acting on $(x, y)^\mathsf{T}$?

- **A)** Because translations are not invertible.
- **B)** Because every 2×2 matrix maps the origin to itself, but a translation moves the origin.
- **C)** Because translations change the area of the image.
- **D)** Because 2×2 matrices cannot contain negative entries.

Q5. The homogeneous vectors $(6, 4, 2)^\mathsf{T}$ and $(3, 2, 1)^\mathsf{T}$ represent

- **A)** the same 2D point $(3, 2)$.
- **B)** the same 2D point $(6, 4)$.
- **C)** two different points, $(6, 4)$ and $(3, 2)$.
- **D)** nothing: $(6, 4, 2)^\mathsf{T}$ is invalid because its last entry is not 1.

Q6. `translate_image(img, -50, 30)` keeps the original canvas size. Where do the black strips appear?

A) left and bottom **B)** left and top **C)** right and top **D)** right and bottom

Q7. The inverse of the translation matrix $T(t_x, t_y)$ is

A) $T(1/t_x, 1/t_y)$ **B)** $T(t_y, t_x)$ **C)** $T(t_x, t_y)^\mathsf{T}$ **D)** $T(-t_x, -t_y)$

C. Scaling and interpolation

Q8. $S = \mathrm{diag}(2, 2, 1)$ applied to the point $(30, 10)$ gives

A) $(32, 12)$ **B)** $(60, 20)$ **C)** $(15, 5)$ **D)** $(60, 10)$

Q9. Which point does the scaling $S(3, 3)$ leave unchanged?

A) The origin $(0, 0)$, i.e. the top-left corner.
B) The centre of the image.
C) The bottom-right corner.
D) None: every point moves.

Q10. `scale_image_warp(img, 2.5, 2.5)` is applied to an image with `img.shape == (400, 600, 3)`. The result has shape

A) `(400, 600, 3)`
B) `(1500, 1000, 3)`
C) `(160, 240, 3)`
D) `(1000, 1500, 3)`

Q11. During a warp, an output pixel maps back to the source position $(40.4,\ 12.8)$. With bilinear interpolation, what weight does the source pixel $(x, y) = (40, 13)$ receive?

A) 0.12 **B)** 0.32 **C)** 0.48 **D)** 0.08

D. Rotation

Q12. With the notebook's rotation matrix and OpenCV's downward y-axis, `rotate_image(img, 10)` makes the image appear

A) rotated 10° counter-clockwise about its centre.
B) rotated 10° clockwise about its top-left corner.
C) rotated 10° clockwise about its centre.
D) rotated 10° counter-clockwise about its top-left corner.

Q13. What does `rotate_image(img, 90)` show?

A) An (almost) entirely black image, because $x' = -y \leq 0$ for every pixel.
B) The image rotated by 90°, neatly filling the canvas.
C) The image flipped upside down.
D) An error, because 90° is not allowed.

Q14. Which matrix is the inverse of the rotation $R(\theta)$?

A) $R(\theta)$ itself **B)** $-R(\theta)$ **C)** $R(1/\theta)$ **D)** $R(\theta)^\mathsf{T} = R(-\theta)$

Q15. A student forgets `np.deg2rad` and writes `np.cos(10)` for a 10° rotation. What is computed?

A) $\cos 10^\circ \approx 0.985$, because NumPy assumes degrees.
B) Nothing: NumPy raises an error.
C) The cosine of 10 radians ($\approx 573^\circ$), about -0.839.
D) $\cos(0.1745)$, because NumPy converts automatically.

Q16. In the 2×2 rotation block, the second column $(-\sin\theta,\ \cos\theta)^\mathsf{T}$ is

A) the image of the vector $(1, 0)$.
B) the image of the vector $(0, 1)$.
C) the centre of the rotation.
D) the translation part of the transform.

E. Shear

Q17. A horizontal shear with $k = 0.5$ is applied to an image 400 px wide and 300 px tall. Where does the bottom-right corner $(400, 300)$ go?

A) $(550, 300)$ **B)** $(400, 500)$ **C)** $(600, 300)$ **D)** $(550, 450)$

Q18. Which of these does a shear preserve?

A) Lengths and angles.
B) Angles only.
C) Area, straight lines and parallelism.
D) Nothing at all.

Q19. After a horizontal shear with $k = 0.3$, the angle at the top-left corner between the top edge and the left edge is approximately

A) 90° **B)** 16.7° **C)** 106.7° **D)** 73.3°

F. The affine family

Q20. A general 2D affine transformation has how many degrees of freedom, and how many point correspondences determine it?

A) 4 and 2 **B)** 6 and 3 **C)** 8 and 4 **D)** 9 and 5

Q21. Which property is preserved by *every* affine transformation?

A) parallelism **B)** lengths **C)** angles **D)** area

Q22. An affine map sends $(0, 0) \mapsto (1, 1)$, $(1, 0) \mapsto (3, 1)$ and $(0, 1) \mapsto (1, 4)$. Where does it send $(1, 1)$?

A) $(4, 5)$ **B)** $(3, 5)$ **C)** $(3, 4)$ **D)** $(2, 3)$

G. Perspective transformation (homography)

Q23. A homography equals the identity except for $g = 0.002$ (bottom-left entry). Where does it send $(500, 100)$?

A) $(500, 100)$ **B)** $(1000, 200)$ **C)** $(501, 100)$ **D)** $(250, 50)$

Q24. Under the same homography, all horizontal lines $y = \text{const}$ converge towards

A) $(0, 0)$ **B)** $(500, 0)$ **C)** $(250, 0)$ **D)** nowhere

Q25. Why does a homography have 8 degrees of freedom instead of 9?

A) Its bottom-right entry must always be 0.
B) OpenCV can only store 8 numbers.
C) H and λH ($\lambda \neq 0$) give exactly the same mapping, so the overall scale is arbitrary.
D) Its determinant must equal 1.

Q26. What is the minimum input that `cv2.getPerspectiveTransform` needs?

A) 4 point pairs, no 3 of them collinear.
B) 3 point pairs.
C) 2 point pairs.
D) 8 point pairs.

Q27. In the DLT system for a homography, each point correspondence $(x, y) \leftrightarrow (u, v)$ provides

A) 1 equation **B)** 2 linear equations **C)** 3 equations **D)** 4 equations

Q28. Which statement is true for a general homography but false for every affine map?

A) Straight lines stay straight.
B) The origin stays fixed.
C) The area of every region is preserved.
D) Parallel lines may meet at a finite vanishing point.

Q29. In `perspective_transform`, the four source corners (top-left, top-right, bottom-left, bottom-right) are sent, in that order, to

```
dst_pts = [[50, 50], [w-50, 0], [0, h-50], [w, h]].
```

Which corner is pushed diagonally inward?

A) top-right **B)** bottom-left **C)** top-left **D)** bottom-right

H. Composing transformations

Q30. Applying A first and then B to a point is the same as applying the single matrix

A) $B \cdot A$ **B)** $A \cdot B$ **C)** $A + B$ **D)** $A \odot B$

Q31. Let $\theta = 10°$ and $\mathbf{t} = (40, -30)$. The third column of $R(\theta)\,T(\mathbf{t})$ (translate first, then rotate) is approximately

A) $(40, -30)$ **B)** $(34.2, -36.5)$ **C)** $(44.6, -22.6)$ **D)** $(0, 0)$

Q32. Which matrix rotates by θ about a point $\mathbf{c}$ (instead of the origin)?

A) $R(\theta)\,T(\mathbf{c})$ **B)** $T(\mathbf{c})\,R(\theta)\,T(-\mathbf{c})$ **C)** $T(-\mathbf{c})\,R(\theta)\,T(\mathbf{c})$ **D)** $T(\mathbf{c})\,T(-\mathbf{c})\,R(\theta)$

Q33. Uniform scaling by 3 about the point $(100, 50)$. What is the third column (t_x, t_y) of its matrix?

A) $(100, 50)$ **B)** $(300, 150)$ **C)** $(-100, -50)$ **D)** $(-200, -100)$

Q34. $(B\,A)^{-1}$ equals

A) $A^{-1}B^{-1}$ **B)** $B^{-1}A^{-1}$ **C)** $B\,A$ **D)** $(A\,B)^{\mathsf{T}}$

Q35. `create_transformation` computes `T = t @ R @ Skew @ Scale`. Which operation acts on a point first?

A) translation **B)** scaling **C)** rotation **D)** skew

Q36. Why is "multiply the matrices, then warp once" better than calling `warpAffine` several times in a row?

A) Because matrix multiplication is commutative, so the order no longer matters.
B) Because `warpAffine` can only be called once per image.
C) It interpolates only once and never loses pixels cropped by an intermediate canvas.
D) It is the only way to apply a perspective transform.

Q37. In NumPy, which expression is the matrix product of `A` and `B`?

A) `A * B` **B)** `A ** B` **C)** `A & B` **D)** `A @ B`

I. Warping and the hand-written loop

Q38. Why do `cv2.warpAffine` and `cv2.warpPerspective` use inverse (backward) mapping?

A) So that every output pixel receives exactly one value: no holes and no collisions.
B) Because inverting a matrix is cheaper than multiplying by it.
C) Because it removes the need for interpolation.
D) Because it keeps the colours in B, G, R order.

Q39. `apply_transformation` uses `p = [[i], [j], [1]]`, where `i` is the row index and `j` the column index. Consequently, a pure translation with `tx = 200`, `ty = 150` would move the image

A) 200 px right and 150 px down.
B) 200 px down and 150 px right.
C) 200 px up and 150 px left.
D) nowhere: translations are ignored by the loop.

Q40. In `create_transformation(angle=0.81, ...)`, the rotation angle is

A) 0.81°.
B) 81°.
C) about 46.4°, because NumPy's `cos/sin` work in radians.
D) 0.81 full turns.

Q41. If forward mapping (as in `apply_transformation`) is used to enlarge an image by a factor of 2, the result typically shows

A) a perfectly smooth enlarged image.
B) swapped colour channels.
C) a smaller image.
D) a regular pattern of black, unfilled pixels (holes).

Part III

Solutions

Answer key

1 **C**	2 **A**	3 **D**	4 **B**	5 **A**	6 **C**	7 **D**
8 **B**	9 **A**	10 **D**	11 **C**	12 **B**	13 **A**	14 **D**
15 **C**	16 **B**	17 **A**	18 **C**	19 **D**	20 **B**	21 **A**
22 **C**	23 **D**	24 **B**	25 **C**	26 **A**	27 **B**	28 **D**
29 **C**	30 **A**	31 **C**	32 **B**	33 **D**	34 **A**	35 **B**
36 **C**	37 **D**	38 **A**	39 **B**	40 **C**	41 **D**	

Worked solutions

A. Images and coordinates

Q1 **C** `shape` is $(H, W, \text{channels})$, so $H = 480$ and $W = 640$. NumPy indexes row first, so $(x, y) = (10, 20)$ is `img[20, 10]`. Option D is wrong: OpenCV uses B, G, R order.

Q2 **A** OpenCV size arguments are `(width, height)`, the reverse of `img.shape`: `dsize = (img.shape[1], img.shape[0])`.

Q3 **D** `cv2.imread` stores channels as B, G, R; Matplotlib interprets the last axis as R, G, B. Without the conversion, red and blue are swapped (a red apple looks blue).

B. Homogeneous coordinates and translation

Q4 **B** $A \cdot (0,0)^\mathsf{T} = (0,0)^\mathsf{T}$ for every 2×2 matrix A, but a translation sends the origin to $(t_x, t_y) \neq (0,0)$. Translations are invertible (A is false), preserve area (C is false), and matrices may contain negative numbers (D is false).

Q5 **A** Divide by the last coordinate: $(6/2,\ 4/2) = (3,2)$ and $(3/1,\ 2/1) = (3,2)$. Homogeneous vectors that differ by a non-zero factor represent the same point; the last entry does not have to be 1.

Q6 **C** $t_x = -50$ shifts the image 50 px *left*, so the rightmost 50 columns of the canvas receive no source pixel (black strip on the right). $t_y = +30$ shifts it 30 px *down* (y points down), so the top 30 rows are black.

Q7 **D** Undoing a shift by $\mathbf{t}$ means shifting by $-\mathbf{t}$: $T(t_x, t_y)\, T(-t_x, -t_y) = T(0,0) = I$.

C. Scaling and interpolation

Q8 **B** Row 1: $2 \cdot 30 + 0 \cdot 10 + 0 \cdot 1 = 60$. Row 2: $0 \cdot 30 + 2 \cdot 10 + 0 \cdot 1 = 20$. The point lies in the same direction from the origin, twice as far away.

Q9 **A** $S(3,3)\,(0,0,1)^{\mathsf{T}} = (0,0,1)^{\mathsf{T}}$, and any other point has its distance from the origin tripled. Scaling about the centre would need the pivot trick $T(\mathbf{c})\,S\,T(-\mathbf{c})$.

Q10 **D** `new_size = (int(600*2.5), int(400*2.5)) = (1500, 1000)` is `(W, H)`; the returned array has `shape (H, W, 3) = (1000, 1500, 3)`.

Q11 **C** $\alpha = 0.4$, $\beta = 0.8$. The pixel $(40, 13)$ is the "left, bottom" neighbour, so its weight is $(1-\alpha)\,\beta = 0.6 \cdot 0.8 = 0.48$. The four weights are $0.12,\ 0.08,\ 0.48,\ 0.32$ and sum to 1.

D. Rotation

Q12 **B** The matrix has a zero third column, so the pivot is the origin (top-left corner). With y pointing down, the formula's positive direction (from $+x$ towards $+y$) is clockwise on screen.

Q13 **A** For $\theta = 90°$: $x' = x\cos\theta - y\sin\theta = -y$ and $y' = x$. Since $y \geq 0$ for every pixel, $x' \leq 0$: everything lands on or left of the canvas edge. Only the column $x' = 0$ (the original top row) can survive.

Q14 **D** Rotation matrices are orthogonal: $R^{\mathsf{T}}R = I$, so $R^{-1} = R^{\mathsf{T}}$. Transposing swaps the signs of the sin entries, which is exactly $R(-\theta)$ (rotating back).

Q15 **C** NumPy's trigonometric functions take radians. 10 rad $= 10 \cdot 180/\pi \approx 573°$ and $\cos(10) \approx -0.839$. The rotation would silently be by the wrong angle.

Q16 **B** Multiply the block by $(0,1)^{\mathsf{T}}$: the result is the second column. In general the j-th column of a matrix is the image of the j-th unit vector.

E. Shear

Q17 **A** $(x, y) \mapsto (x + ky,\ y) = (400 + 0.5 \cdot 300,\ 300) = (550, 300)$. The height never changes in a horizontal shear.

Q18 **C** $\det H_x(k) = 1$, so areas are preserved; like every affine map, a shear keeps straight lines straight and parallel lines parallel. Lengths of non-horizontal segments and angles change.

Q19 **D** The top edge points along $(1, 0)$ and the left edge along $(k, 1)$. The angle between them is $90° - \arctan k = 90° - 16.7° = 73.3°$. ($106.7°$ is the angle at the bottom-left corner.)

F. The affine family

Q20 **B** Six unknowns $(a_{11}, a_{12}, a_{21}, a_{22}, t_x, t_y)$; each correspondence gives two equations, so three non-collinear correspondences determine the map (`cv2.getAffineTransform`).

Q21 **A** The bottom row $(0, 0, 1)$ keeps points at infinity at infinity, so parallel lines stay parallel. Lengths, angles and area are preserved only by special affine maps (e.g. rotations).

Q22 **C** An affine map sends parallelograms to parallelograms, so $D' = B' + C' - A' = (3,1) + (1,4) - (1,1) = (3,4)$. Check: $\mathbf{t} = (1,1)$, the columns are $(2,0)$ and $(0,3)$, so $(1,1) \mapsto (1+2,\ 1+3) = (3,4)$.

G. Perspective transformation (homography)

Q23 **D** $x' = 500$, $y' = 100$, $w' = 0.002 \cdot 500 + 1 = 2$, so the final point is $(500/2,\ 100/2) = (250, 50)$.

Q24 **B** On a line $y = \text{const}$: $x_{\text{final}} = \frac{x}{1+0.002x} \to \frac{1}{0.002} = 500$ and $y_{\text{final}} = \frac{y}{1+0.002x} \to 0$ as $x \to \infty$, independent of y. All horizontal lines meet at the vanishing point $(500, 0)$.

Q25 **C** The final coordinates are ratios x'/w', y'/w', so a common factor λ cancels. One entry can therefore be fixed (usually $h_{33} = 1$), leaving $9 - 1 = 8$ degrees of freedom.

Q26 **A** 8 unknowns and 2 equations per pair: 4 pairs. If three points are collinear the system becomes degenerate (no unique solution).

Q27 **B** $u(gx + hy + 1) = ax + by + c$ and $v(gx + hy + 1) = dx + ey + f$ are two equations, both linear in $a, \ldots, h$.

Q28 **D** Both affine maps and homographies keep straight lines straight (A). Only a homography with $(g, h) \neq (0, 0)$ can map parallel lines to lines that meet at a finite vanishing point.

Q29 **C** The first source corner $(0, 0)$ is the top-left one, and it goes to $(50, 50)$: 50 px right and 50 px down at the same time, i.e. diagonally inward. Top-right moves only left, bottom-left only up, bottom-right moves by about one pixel.

H. Composing transformations

Q30 **A** $B(A\mathbf{p}) = (BA)\mathbf{p}$: in a product, the rightmost matrix acts first. ($A \odot B$ is the element-wise product, which is not a composition.)

Q31 **C** $R\,T(\mathbf{t})$ has third column $R\mathbf{t}$: $(\cos 10° \cdot 40 - \sin 10° \cdot (-30),\ \sin 10° \cdot 40 + \cos 10° \cdot (-30)) \approx (39.39 + 5.21,\ 6.95 - 29.54) = (44.6, -22.6)$. Option A is $T\,R$ (rotate first); option B is what you get by rotating $\mathbf{t}$ the wrong way.

Q32 **B** Read right to left: $T(-\mathbf{c})$ moves $\mathbf{c}$ to the origin, $R(\theta)$ rotates about the origin, $T(\mathbf{c})$ moves it back. Check: $\mathbf{c} \mapsto \mathbf{0} \mapsto \mathbf{0} \mapsto \mathbf{c}$, so $\mathbf{c}$ is fixed. Option C rotates about $-\mathbf{c}$.

Q33 **D** $T(\mathbf{c})\,S\,T(-\mathbf{c})$ has third column $\mathbf{c} - S\mathbf{c} = (1 - s)\,\mathbf{c} = (1 - 3)(100, 50) = (-200, -100)$. Check: $(100, 50) \mapsto (3 \cdot 100 - 200,\ 3 \cdot 50 - 100) = (100, 50)$ stays fixed.

Q34 **A** $(BA)(A^{-1}B^{-1}) = B(AA^{-1})B^{-1} = BB^{-1} = I$. To undo "$A$ then B" you undo B first, then A.

Q35 **B** In `t @ R @ Skew @ Scale` the rightmost factor, `Scale`, multiplies the point first; then `Skew`, `R` and finally the translation `t`.

Q36 **C** Each `warpAffine` call interpolates (adds blur) and crops to its canvas; pixels cropped in an intermediate result are gone and their black fill is moved into view by later warps. Matrix multiplication is *not* commutative (A is false), and `warpAffine` can be called any number of times.

Q37 **D** `@` is matrix multiplication (`np.matmul`). `*` multiplies element by element, `**` is element-wise power and `&` is a bitwise AND.

I. Warping and the hand-written loop

Q38 **A** Looping over output pixels and asking "where did I come from?" (via M^{-1}) assigns exactly one interpolated value to every output pixel. Interpolation is still needed (C is false).

Q39 **B** The first component of `p` is the row index, so `tx` is added to the row (vertical, downwards) and `ty` to the column (horizontal, to the right).

Q40 **C** The angle is used directly in `np.cos`/`np.sin`, so it is in radians: $0.81 \cdot 180/\pi \approx 46.4°$. (`rotate_image` converts from degrees; this function does not.)

Q41 **D** Forward mapping sends each of the $H \cdot W$ source pixels to one destination pixel, but the enlarged image has $4HW$ pixels; about three out of four are never written and stay black. Inverse mapping avoids this.

Formula sheet

Transform	Matrix	$(x, y) \mapsto$	Inverse	DOF
Translation	$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$	$(x + t_x,\ y + t_y)$	$T(-t_x, -t_y)$	2
Scaling	$\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$(s_x x,\ s_y y)$; fixed point $(0,0)$	$S(\frac{1}{s_x}, \frac{1}{s_y})$	2
Rotation	$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$(x\cos\theta - y\sin\theta,$ $x\sin\theta + y\cos\theta)$; clockwise on screen for $\theta > 0$	$R(\theta)^\mathsf{T} = R(-\theta)$	1
Shear (x)	$\begin{bmatrix} 1 & k & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$(x + ky,\ y)$; $\det = 1$; corner angle $90^\circ - \arctan k$	$H_x(-k)$	1
Shear (y)	$\begin{bmatrix} 1 & 0 & 0 \\ k & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$(x,\ y + kx)$	$H_y(-k)$	1
Affine	$\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$	$A\mathbf{p} + \mathbf{t}$; keeps parallel lines; 3 point pairs	$\begin{bmatrix} A^{-1} & -A^{-1}\mathbf{t} \\ \mathbf{0}^\mathsf{T} & 1 \end{bmatrix}$	6
Homography	$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix}$	$\left(\frac{ax+by+c}{gx+hy+1}, \frac{dx+ey+f}{gx+hy+1}\right)$; 4 point pairs	H^{-1}	8

Homogeneous coordinates

$(x, y) \to (x, y, 1)^\mathsf{T}$; $(X, Y, W)^\mathsf{T} \to (X/W,\ Y/W)$; $(X, Y, W)^\mathsf{T} \sim \lambda(X, Y, W)^\mathsf{T}$. Affine: bottom row $(0, 0, 1)$, so $w' = 1$. A direction (d_x, d_y) is the point at infinity $(d_x, d_y, 0)^\mathsf{T}$.

Composition

First A, then B: BA (right to left).
$(BA)^{-1} = A^{-1}B^{-1}$.
$T(\mathbf{t})\,R = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^\mathsf{T} & 1 \end{bmatrix}$, $R\,T(\mathbf{t}) = \begin{bmatrix} R & R\mathbf{t} \\ \mathbf{0}^\mathsf{T} & 1 \end{bmatrix}$.
About a point $\mathbf{c}$: $T(\mathbf{c})\,X\,T(-\mathbf{c}) = \begin{bmatrix} X & \mathbf{c} - X\mathbf{c} \\ \mathbf{0}^\mathsf{T} & 1 \end{bmatrix}$.
Scale by s about $\mathbf{c}$: third column $(1 - s)\,\mathbf{c}$.

Homography (DLT)

Fix $h_{33} = 1 \Rightarrow 8$ unknowns. Per pair $(x, y) \leftrightarrow (u, v)$:

$$ax + by + c - gxu - hyu = u$$
$$dx + ey + f - gxv - hyv = v$$

4 pairs (no 3 collinear) $\Rightarrow 8 \times 8$ linear system.

Hierarchy (DOF)

rigid (3) $\subset$ similarity (4) $\subset$ affine (6) $\subset$ projective (8).

Bilinear interpolation at $(x_0 + \alpha,\ y_0 + \beta)$

$$(1-\alpha)(1-\beta)I_{00} + \alpha(1-\beta)I_{10} + (1-\alpha)\beta I_{01} + \alpha\beta I_{11}$$

Mapping

Forward ($\mathbf{p} \mapsto M\mathbf{p}$): holes and collisions.
Inverse ($\mathbf{q} \mapsto M^{-1}\mathbf{q}$, then interpolate): one value per output pixel; used by OpenCV.

Conventions

Pixel (x, y) is `img[y, x]`; `shape` $= (H, W, 3)$; `dsize` $= (W, H)$; channels B, G, R.
y points down, so $R(\theta)$ with $\theta > 0$ looks clockwise.
`np.cos`, `np.sin` use radians: `np.deg2rad(deg)`.
Matrix product in NumPy: `A @ B` (not `A * B`).

OpenCV

`warpAffine(img, M[:2], (W, H))`
`warpPerspective(img, H, (W, H))`
`getAffineTransform(src3, dst3)`: 3 pairs
`getPerspectiveTransform(src4, dst4)`: 4 pairs
`findHomography(src, dst, cv2.RANSAC)`: ≥ 4
`getRotationMatrix2D(c, deg, s)`: deg > 0 is counter-clockwise on screen